AI Engineer, Sourcing Agent Take-Home

Build a bounded agent that searches for people, learns from what it finds, and knows when to stop. · 2-3 hours · GloPros

> **Time box: 2-3 hours.** We mean it. We would rather read a small loop that stops for the right reason than a large one that never stops. If you run out of time, stop and write down what you would have done next; that section counts for real marks.

The scenario

Screening is the easy half of hiring. The hard half is **sourcing**: nobody has applied yet, and something has to go out and find people who have never heard of the job.

Our recruiters do this by searching a candidate database, reading what comes back, realising their search was wrong, and searching again. That loop is the product. We want to know whether you can build an agent that runs it, and, more to the point, one a recruiter would trust to run unattended.

> **The one sentence that matters.** A recruiter will forgive an agent that finds fewer people than hoped. They will not forgive one that quietly changed the brief in order to look successful.

What you get

File	Contents
`sourcing_provider.py`	The search API. Your agent must go through it. Quota, rate limits and faults are documented in its docstring.
`example_run.py`	A working call, with the retry you will need. Run it first.
`data/profiles.jsonl`	The 2,500 profiles the provider searches. Read it to understand the data, though your agent must still go through the provider, and the audit log shows whether it did.
`data/vacancies.json`	4 vacancies: description, `hard_requirements`, `rubric`, `target_strong`.
`data/gold_relevance.json`	Recruiter labels for **2 of the 4** vacancies. We hold the other two.
`data/output_schema.json`	Schemas for what you must produce.

What to build

An agent that, for each of the four vacancies, runs a loop:

search → score what came back → work out what the result means → adjust → search again → stop.

And then tells a recruiter what it found and what it concluded.

The two product rules

These are not suggestions. They are what makes the difference between a tool a recruiter uses and one they turn off.

> **1. Hard requirements are never relaxed.** Each vacancy has a `hard_requirements` block: location, minimum years, full-time, no visa sponsorship. Those came from the hiring manager. Your agent may *propose* relaxing one, in its report, with its reasoning. It may never *do* it. We check every search call you made against the stated requirements.
>
> We have watched this go wrong for real: a search quietly widened past the stated city and returned 586 people on the wrong continent, and the recruiter found out by reading the list.
>
> **2. Every adjustment must come with evidence a human can check.** "The model suggested adding TypeScript" is not evidence. "9 of the 11 people who passed list TypeScript; 3 of the 47 who failed do" is. We read the `evidence` field on every iteration.

The five constraints

We are deliberately **not** telling you how to structure this. No prescribed components, no required libraries, no diagram to copy. What we are giving you is the set of things that must be true of whatever you build, chosen so that only a sound design satisfies all five.

1. **It survives being killed.** We will `kill -9` your agent mid-run and start it again. It must carry on without re-spending quota on work it already did.
2. **It is observable while it runs.** Someone watching must be able to tell which vacancy it is on, which iteration, and why it is still going.
3. **It stops cleanly when asked.** Cancel it and it keeps what it has found so far.
4. **A misbehaving provider is normal, not an outage.** Rate limits, 502s, timeouts and an exhausted quota are all expected conditions with correct handling, not stack traces.
5. **The loop terminates, and says why.** Every vacancy ends with a `stop_reason` that is true.

Your README explains the shape you chose and why it satisfies these. That explanation is what we are actually grading in this part.

Where the LLM goes, and where it does not

At least one step must use an LLM. Which steps, and why, is the interesting decision, and your README must state it explicitly: **what you gave to a model, what you kept in code, and why that line is where it is.**

We have a strong opinion here and we are not going to tell you what it is before you have formed yours.

The targets

Each vacancy carries a `target_strong`, how many strong candidates the recruiter asked for.

The budget

Deliberately tight, because sourcing quota is the real constraint in this product.

- **1,500 profiles for the entire run**, shared across all four vacancies. It does not reset. Spending it all on the first vacancy is a decision, and it will show.
- `count()` costs no quota. `search()` costs one unit per profile returned.
- A full cold run: **under 10 minutes** and **under $3** of API spend. Measure both. Report both.

Getting an LLM

Reply to the email this task came with and we will send you an API key with enough budget for roughly twenty full runs. Using your own key, or a local model, is equally fine; just tell us which model you used.

No access at all? Say so, build against a stub interface, and prove one real call in a test. You will not be marked down for a provider you could not reach.

Practicalities

- **Python.** Any libraries, any LLM provider, any storage. The provider stub needs no network and no install beyond the standard library.
- **AI coding assistants are allowed and expected.** We use them daily. Two conditions: you can explain and defend every line, and your README carries a short **AI assistance** section saying which tools you used, what they got wrong, and what you rewrote. Nobody has lost points for that section.
- **Your agent will meet vacancies it has never seen.** Read them from the file. Nothing that only works for these four belongs in the agent.
- **Do not spend time on** Docker, CI, auth, or a UI. None of it is marked.

What to send back

A git repository with:

1. **Your code.**
2. `out/trace.jsonl`, one line per iteration per vacancy. Schema in the pack.
3. `out/shortlist.json`, up to 25 ranked profiles per vacancy, all four present, with a reason and verbatim evidence for each.
4. `out/report.md`, per vacancy: what it did, why it stopped, and **what you would tell the recruiter.** Write that last part as if the recruiter is the reader, because that is the actual product.
5. `.provider_audit/`, committed. This is the provider's own record of every call your agent made. We read it, and where it disagrees with your trace, it wins.
6. `README.md`, under two pages:

Section	What goes in it
Run it	Exact commands, clean clone to output files.
Design	The shape you chose, and how it meets the five constraints.
LLM boundary	What the model decides, what code decides, why.
Findings	What you would tell a recruiter about each of the four vacancies.
Weaknesses	Where it breaks. Specific. This section earns marks.
Next	What you would do with another week, in priority order.
AI assistance	Which assistants you used, what they got wrong, what you rewrote.

Commit as you go. We read `git log`, not to count commits, but to see whether the system grew or arrived.

Optional, only if the clock allows

One only, and an unfinished one is worse than none. If you are short of time, skip this and write a better report instead.

Outreach with a human gate. For the top three on one vacancy, draft a personal opening message. Nothing may reach a send function unless a human has approved *that exact text*. Approving a draft and then sending a differently-generated message is the failure mode we care about. Sending is out of scope; the gate is the point.

How we assess it

Half the grade is what you send. The other half is the debrief.

What you send, 50%

Area	Share	The question behind it
Loop quality and termination	25%	Does it improve on what it learned, and stop for a true reason?

Area	Share	The question behind it
What it tells the recruiter	20%	Is the answer honest and useful, above all where things did not go to plan?
Architecture and resilience	20%	Does the design actually satisfy the five constraints?
Efficiency under quota	15%	Strong candidates found per profile spent.
Grounding and safety	10%	Hard requirements respected; evidence real; explanations true.
Communication	10%	Can we follow the thinking without you in the room?

The debrief, 50%

Area	Share	The question behind it
The unseen pack	40%	On people and vacancies it was not built for, does it still work, and still tell the truth?
The live change	30%	Can you change your own system quickly and safely, and show the change did what you meant?
Your own code	30%	Can you explain why it is built the way it is, and where it breaks?

For calibration: a control agent we wrote (one search per vacancy, three pages, no refinement, no diagnosis) spends **367 profiles** and finds **13 strong candidates**, at **3.5 strong per 100 profiles**. It also fails every one of the five constraints. Beating it on raw numbers is not the hard part.

Afterwards

A **45-minute debrief**, and it is half the grade.

- **Before it, we run your agent on a second pack you have not seen.** Same provider API, same file formats. Different people, different vacancies with different ids, and an upstream that is a little less reliable. We bring what your agent did there to the call.
- **In it, we read both reports with you,** ask you to walk us through parts of your code, and give you a small change to make live in your own repo while we watch. Assistants are allowed; that is the point.

Build it so that you could change it in front of someone a week later, on data it has never seen.

FAQ

Can I read `profiles.jsonl` directly instead of searching? To understand the data, yes. To produce your shortlist, no. The audit log records what the agent actually fetched, and a great shortlist with an empty audit log answers a different question than the one we asked.

How many iterations should the loop run? That is the question, not a setting we forgot to tell you.

What if an LLM call fails? Same as a provider call failing. It is a normal condition.

Can I change the output schemas? Add fields freely. Do not rename or remove the required ones. Our scorer reads them.

Am I allowed to go over three hours? Allowed, discouraged, and please tell us honestly how long you spent. We read the answer as information about the task, not about you.